Arquitecturas

1.Arquitecturea en capas

¿QUE ES ?
- Divide la aplicación en capas o niveles, cada uno con responsabilidades específicas. Esto facilita la organización, el mantenimiento y la escalabilidad.

Características:
- Separa las responsabilidades del sistema.
- Cada capa se comunica con la capa superior o inferior.
- Permite realizar cambios sin afectar toda la aplicación.
- Facilita la organización del código.

Patrones de diseño:
- Singleton: garantiza que una clase tenga una única instancia. Ejemplo: una conexión a la base de datos.
- Factory Method: permite crear objetos sin especificar directamente la clase exacta que se utilizará

2. Arquitectura monolítica

¿Qué es?
Toda la aplicación se encuentra en un solo bloque o unidad. Sus componentes están conectados y se despliegan juntos.

Características:
- Los módulos comparten el mismo entorno.
- Es sencilla de desarrollar al inicio.
- Los componentes están integrados.
- Puede ser difícil de escalar cuando crece el proyecto.

Patrones de diseño:
- Abstract Factory: crea familias de objetos relacionados sin especificar sus clases concretas.
- Builder: construye objetos complejos paso a paso.
- Prototype: permite crear nuevos objetos copiando instancias existentes.

3. Arquitectura cliente-servidor

Características:
- Separa el frontend del backend.
- Utiliza protocolos de comunicación como HTTP y TCP.
- Un servidor puede atender a varios clientes.
- Puede organizarse en dos, tres o más capas.
- Centraliza los datos y facilita su administración.

¿Qué es?
Es un modelo en el que el cliente solicita información o servicios y el servidor procesa la petición, consulta la base de datos cuando es necesario y devuelve una respuesta.

Patrones de diseño:
- Adapter: permite que dos sistemas con interfaces diferentes puedan comunicarse.
- Facade: ofrece una interfaz sencilla para utilizar un conjunto de componentes o subsistemas.

4. Arquitectura MVC

Componentes:
- Modelo: administra los datos y la lógica del negocio.
- Vista: representa la interfaz gráfica con la que interactúa el usuario.
- Controlador: recibe las solicitudes y coordina la comunicación entre el modelo y la vista.

Características:
- Organiza mejor el código.
- Facilita el trabajo en equipo.
- Mejora el mantenimiento de la aplicación.
- Puede generar más archivos y requiere una buena organización.

Patrones de diseño:
- Decorator: añade funcionalidades a un objeto sin modificar su código original.
- Proxy: actúa como intermediario y controla el acceso a un objeto o recurso.
- Bridge: separa la abstracción de la implementación para que puedan modificarse de manera independiente.

¿Qué es?
Es un patrón de arquitectura que divide la aplicación en tres componentes: Modelo, Vista y Controlador. Cada uno tiene una responsabilidad específica.